Andres Trujillo

Oct 25, 2026

CS 470 Final Reflection

https://youtu.be/JxorXphqk8Q



Experiences & Strengths

Skills I have developed include Docker containerization and coordination with local networking concepts applied to docker containers as command line arguments or docker files. I revisited core MongoDB concepts such as collections, database security and permissions, front end requirements for connectivity, and backend query optimization techniques such as indexing or being using Atlas to scale a web application. I've learned how to use AWS cloud resources such as S3 Buckets, DynamoDB, and API Gateway's to route requests appropriately using AWS Lambda functions. These skills are invaluable to a software engineer because they are tools used

to solve business problems and help to create robust, easy to document solutions used by developers across the world.

My strengths as a developer is being able to synthesize information from documents quickly because I understand fundamental programming principles such as keeping code clean, reliable and functional. These fundamentals coupled with algorithmic knowledge help me effectively choose the right technology for real world use cases. In this instance migrating from local containers to serverless proved that having a wide breadth of knowledge allows a developer to understand the system for which he may deploy a solution.

The types of roles that I was preparing for was DevOps, where managing the development operational pipeline such as having our web application find gateways and manage the architecture of the framework would allow us to easily jump our web application to a different system while being able to scale out for our userbase. Web applications in the modern era use NodeJS heavily so being able to code within the NodeJS environment is essential for todays technological solutions and processes within the computer science field.

Planning for Growth

Our initial monolithic architecture or containerized deployment transition to a serverless and microservices architecture unlocks greater efficiencies in both operational management and horizontal scaling capabilities.

When we have moved to lambda functions from a microservice that handles http requests by using the Amazon API Gateway our requests scale automatically. Which means if we have a lot of users using our web app we have many micro-vm's handling the increased user base. This serverless capability is managed by Amazon instead of having complicated deployment

environments for docker containers. One is unmanaged and the latter is completely our responsibility to ensure uptime and availability.

Error handling is logged via other serverless and Amazon controlled services. This enables our web application to log its error and retry using a different server without manual intervention. The direct process of reading the errors comes from structured JSON responses enabling programmatic resolution by software developers or instant acess to the data of our errored API calls. There is also Amazon SAM that can mimic on our machine the exact environment that created the error and this would eventually lead to much faster bug development. Sometimes managing your own docker container could lead to multiple logs and multiple events being cascaded but Amazon services are capable of catching the error quickly and providing the detailed information discussed to further investigate the cause and origin.

Cost prediction in a serverless environment is based upon CPU time, memory allocation, and total invocation volume over a set period. That allows granular insight into how web application is being used, and whether it is being used incorrectly. Another aspect is that database read and writes are billed in the exact way. If you have a popular site with many function calls like a NodeJS game you would have a much, much higher bill than to a simple blog website where you only catalogue your adventures or technological endeavors. The tradeoff with dockers versus serverless are that one is costs predictable because the more containers deployed means the more compute power you must provision. And the trade off made is that serverless offers cost efficiency for unpredictable traffic. So understanding the size of your userbase is very important for both solutions. Surges can be handled more gracefully with serverless and managing your own servers allows you to not be billed for any accidental bugs that consume lots of compute power increasing your bill.

There are several pros and cons to both solutions for any web application. Serverless eliminates a lot of security maintenance that must occur such as staying up to date with zero day vulnerabilities and patching servers. This happens automatically through Amazon managed services but not local servers. Serverless functions also may initially take some time to fire up due to a time period where there is low consumption to a moment of where there is huge surge in users at the same time. This could happen with in-app events or align with business use cases such as coffeeshops early in the morning or during a sunday brunch rush. The extra customers to a locally managed server would require this capability upfront as an initial server cost but with Amazon they scale up with the drawback being slow startup times. This is known as cold start latency.

Elasticity ensures that the application dynamically adapts to fluctuating user demand without manual intervention. This is the primary economic and technical driver for modern cloud expansion because for local servers you would need a dedicated team handling this for you which may increase the cost of your total operation, in comparison to the Amazon billing model. So in theory without wild surges and unpredictable usage for startups this means an efficient and lean business model that scales linearly with userbase consumption.